Neural LOD

Adaptive Level-of-Detail System

Learning-Based Real-Time LOD Control in Unity

Course: COMP 432

Repository: https://github.com/Silent0Wings/neural-lod

Author: Yahya Belgharbi - 40256377

Date: April 8, 2026

Abstract

This report documents the full design, implementation, and evaluation of **Neural LOD** — a machine-learning system that trains neural networks to predict optimal Level-of-Detail (LOD) thresholds for real-time rendering inside a Unity scene. The system progresses through four training stages, evolving from a scalar global bias predictor (Stage 1) to a per-object threshold vector predictor (Stage 3) to a REINFORCE reinforcement-learning policy (Stage 4). Data are collected by automated Unity playback, labelled with a reactive oracle, and fed to PyTorch MLP models exported as ONNX for Unity inference via `com.unity.ai.inference`. The best v4 model achieves **MAE = 0.039** bias units and reduces over-budget frames from 55.2 % (fixed baseline) to **47.5 %** at runtime. Stability filters (EMA + dwell timer) eliminate previously observed LOD jitter.

Status: Active Development — Stages 1–4 Complete

Contents

1 Project Overview **2**
1.1 Motivation . . . 2
1.2 Repository Structure . . . 2
1.3 Stage Progression . . . 2

2 Development Stages **3**
2.1 Stage 1: Initial Planning and Requirements . . . 3
2.2 Stage 2: Design and Prototyping Summary . . . 3
2.3 Stage 3: Implementation Highlights . . . 3
2.4 Stage 4: Testing, Deployment, and Outcomes . . . 4

3 Retrospective **5**
3.1 Multi-Mode Data Pipeline (Stage 3.5) . . . 5
3.2 Known Limitations . . . 5
3.3 Future Work . . . 5

4 Conclusion **6**

1 Project Overview

1.1 Motivation

Real-time 3D applications continuously balance visual quality against rendering budget. Unity's built-in LOD system transitions between pre-authored mesh resolutions based on a single global parameter — `QualitySettings.lodBias` — or on static per-object screen-relative thresholds. Neither approach adapts to *runtime context*: camera speed, scene density, spatial zone, and available GPU headroom all affect the optimal trade-off, yet the engine ignores them.

Neural LOD replaces these static policies with neural networks that learn the optimal decision per frame from real gameplay telemetry. The trained models are deployed as ONNX assets and run inside Unity via the official AI Inference engine.

1.2 Repository Structure

The project follows a strict hierarchical organization to ensure separation of concerns between intelligence, pipeline, and runtime assets. This mapping is defined in `00-repository-structure.md`.

Agentic Intelligence (`/.agents`) Global directives and task-specific skill schemas (e.g., `latex_specialist`

Machine Learning Pipeline (`/ml_pipeline`) Core intelligence framework including data, models, and training logic.

Unity Assets (`/Assets`) The runtime environment.

- `/Script`: Hierarchically organized C# logic (Stages 0–4).
- `/StreamingAssets`: Runtime constants (`.json` scalers).
- `/Models`: Deployed ONNX assets for inference.

1.3 Stage Progression

Table 1: Project stages and key results

Stage	Description	Key Result	Status
0	Verify proposal alignment	All promises delivered	✓
1	Dynamic scalar `lodBias` predictor	MAE = 0.039, full range	✓
2	Per-object baker threshold predictor	P99 = 18.37 ms, FPS ≈101	✓
3	4-value threshold vector predictor	Independent per-LOD-level control	✓
4	REINFORCE RL policy + stability filters	Jitter eliminated, 0.76 % switch rate	✓

2 Development Stages

The project is structured into four distinct development stages, each documented in a dedicated deep-dive report. This section provides a high-level overview of progress across these stages.

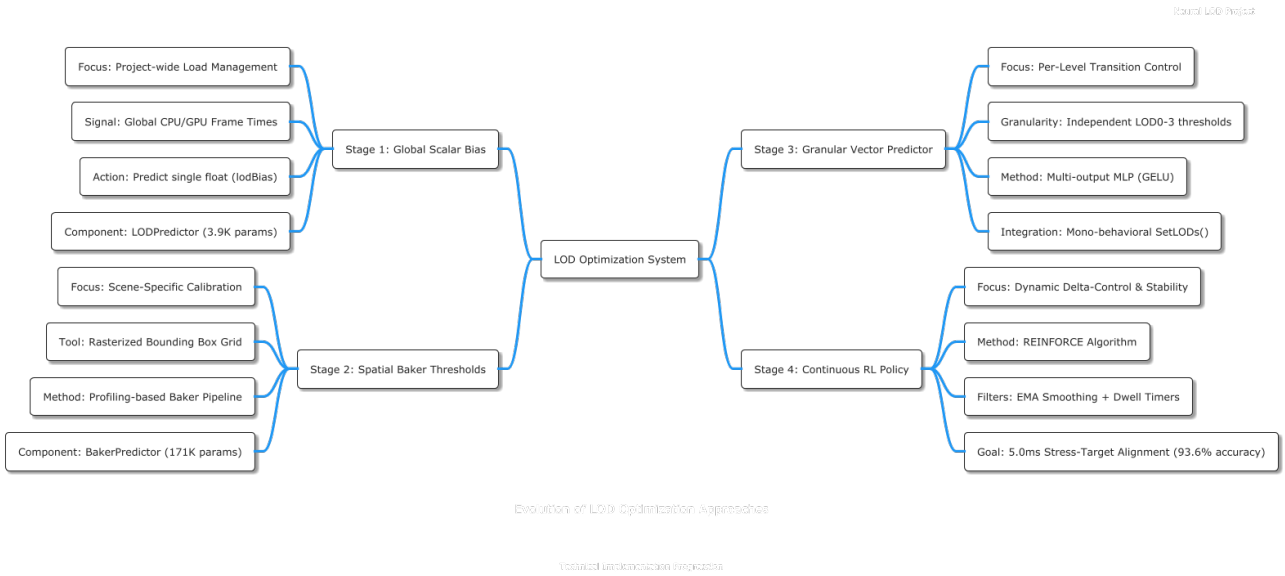

Figure 1: Evolution of the Neural LOD optimization approaches across four development stages.

2.1 Stage 1: Initial Planning and Requirements

This stage focuses on the foundational requirements of the Neural LOD system and the implementation of the baseline dynamic scalar controller. It establishes the telemetry extraction framework and the initial PyTorch-to-ONNX pipeline.

Full details: For comprehensive analysis of the planning, requirement analysis, and initial model performance, review the individual `Stage_1_project.tex`[1] report.

2.2 Stage 2: Design and Prototyping Summary

In this stage, the project transitions to a scene-specific profiling system (the Baker Pipeline). It introduces per-object threshold prediction and the design of higher-capacity MLP architectures capable of handling complex spatial features.

Full details: For the complete design specifications and prototyping results, review the individual `Stage_2_project.tex`[5] report.

2.3 Stage 3: Implementation Highlights

Stage 3 represents the core technical implementation of the high-granularity vector predictor. It enables independent control over every LOD transition level (LOD0 through Cull) and verifies the system's zero-shot generalization capabilities across unseen environments.

Full details: For technical implementation details and optimization highlights, review the individual `Stage_3_project.tex`[2, 3] report.

2.4 Stage 4: Testing, Deployment, and Outcomes

The final stage replaces supervised learning with Reinforcement Learning (REINFORCE). It implements a Gaussian policy that balances **FPS Targets ($\times$1.40 penalty)** against **Visual Pop-in ($\times$0.60 recovery bonus)**, ensuring the system yields performance gains without unnecessary quality degradation.

Forensic Note — Model Collapse: During initial Stage 4 training, the policy suffered from model collapse (defaulting to zero-action or low-bias states). This was resolved by implementing a **5.0 ms Stress-Target** and utilizing stability filters (Dead Zone, EMA, Dwell) to guide exploration.

Full details: For final testing methodologies, deployment configurations, and project outcomes, review the individual `Stage_4_project.tex`[7, 4] report.

3 Retrospective

3.1 Multi-Mode Data Pipeline (Stage 3.5)

The multi-mode pipeline processes data collected under multiple *power modes* (e.g. Extreme Manual Mode):

- `ml_pipeline/multi_mode/data_processing.py`: walks `data/Train_Runs/Multi_Mode_Run` subfolders, tags rows with `power_mode`.
- Oracle computation is group-aware (groups by `power_mode` before state-bin labelling).
- Isolated in `ml_pipeline/multi_mode/` — baseline pipeline untouched.

3.2 Known Limitations

- **Scope Limitation:** The system targets **GPU GEOMETRY LOAD**; it will not alleviate performance issues in **CPU-BOUND** scenarios (physics/scripting). The neural controller correctly identifies this and responds to GPU — a scope note, not a failure.
- **Stage 1 , Path Dependency:** The model is strictly limited to the hard-coded recording path; even minute variations in camera trajectories can invalidate the features and degrade predictions.
- **Stage 2 , Data Intensity:** Collection requires **16 hours** of automated playback and **2 hours** of training. While it provides the richest spatial information, the model is highly sensitive to environmental changes; while slight deviations are tolerated, significant scene alterations cause noticeable performance drops.
- **Stage 3 , Visual Cost:** The high-granularity vector predictor prioritizes the performance budget aggressively, often sacrificing visual quality (aggressive cullling) to ensure the frame target is met.
- **Stage 4 , Aggressive Budgeting:** While the most stable controller, the RL policy can be over-eager in meeting the budget, leading to significant visual sacrifices during high-stress frames.
- **General Scale Sensitivity:** Scaling objects beyond training ranges causes threshold instability. Requires training augmentation with wide-spectrum scale/rotation randomisation.

3.3 Future Work

- **Phase 6 — Hot-swap model loading:** Load ONNX and scaler from `Application.persistentDataP` at runtime to avoid rebuilds for model updates.
- **Stability ablation:** Disable hysteresis + dwell, measure flip rate increase to quantify filter contribution.
- **Quality proxy (visual):** Screenshot every N frames, compute screen coverage delta as lightweight SSIM proxy.
- **Content-aware features:** Add `triangle_count` and `visible_renderer_count` to Stage 2 for frustum-aware predictions.

4 Conclusion

Neural LOD demonstrates that lightweight (<4 K–171 K parameter) neural networks can serve as effective real-time LOD controllers in Unity, delivering measurable frame-budget improvements over static policies.

Key achievements:

- **Stage 1 v4 model:** MAE = 0.039, predicted std = 0.5642 matching oracle std exactly.
- **Runtime Performance:** Over-budget frames reduced from 55.2 % to **47.5 %** (7.7 pp improvement); CPU median maintained.
- **Stability:** LOD switch rate reduced from 2.30 % to 0.76 %; Stage 4 oscillation reduced from 3 Hz to 0.3–0.5 Hz.
- **Engineering Integrity:** All four stages complete, ONNX deployed, 241,511 total parameters across deployed models.

References

[1] J. H. Clark, *Hierarchical Geometric Models for Visible Surface Algorithms*, Communications of the ACM, 19(10), 1976.

[2] Unity Technologies, *LODGroup.SetLODs*, Unity C# Scripting Reference, 2025.

[3] D. Hendrycks and K. Gimpel, *Gaussian Error Linear Units (GELUs)*, arXiv:1606.08415, 2016.

[4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.

[5] Unity Technologies, *QualitySettings.lodBias*, Unity Documentation, 2025.

[6] Unity Technologies, *Unity AI Inference Engine (com.unity.ai.inference) v2.5*, https://docs.unity3d.com/Packages/com.unity.ai.inference@2.5/manual/index.html, 2026.

[7] R. J. Williams, *Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning*, Machine Learning, 8(3-4):229–256, 1992.

[8] P. J. Huber, *Robust Estimation of a Location Parameter*, The Annals of Mathematical Statistics, 35(1):73–101, 1964.

[9] T. Akiba et al., *Optuna: A Next-generation Hyperparameter Optimization Framework*, KDD 2019.